Lesson notes · for you

Data quality and contracts

Data engineering · Level 4 · Lesson 3

Time	Materials
17 min	Your phone or a computer, for the SQL and Python steps; Paper, for writing a contract

Learning goals

- Write quality tests as code: types, nulls, ranges, uniqueness, references, freshness.
- Agree a data contract between producers and users.
- Decide what a failed test does.

Activity: Contract and tests

1. Choose (3 min): Pick a dataset you depend on, produced by someone else.
2. Contract (6 min): Write the columns, types, allowed values, the key and the freshness rule.
3. Test (5 min): Turn three rules into tests that return the bad rows.
4. Decide (3 min): For each test, choose what a failure does, and why.

Swap contracts with a partner and try to write a row that breaks theirs but passes every test.

Discussion

- Why use a LEFT JOIN to find orphans?
- When is quarantine better than blocking the load?
- What can’t any test catch?

Check yourself

- Write one rule of a data contract and the SQL test that checks it.

Try it this week

Write a data contract for a dataset you use, like your class timetable or a club’s results. List the columns, types and allowed values, and who fixes it when it’s wrong.

Worksheet

Data quality and contracts

Data engineering · Level 4 · Lesson 3

Name _____ Class _____ Date _____

1 **A test fails on 3 rows out of 10,000. Which response keeps the pipeline moving and the data honest?**

- ◯ Let them through and say nothing
- ◯ Quarantine the 3 rows and alert the producer
- ◯ Block the whole load until next week

2 **Match each side of quality to a fault that breaks it.**

Valid	• • • •	A mark for a pupil who doesn't exist
Unique	• • • •	A mark of 25 out of 20
Consistent	• • • •	Monday's file still missing on Wednesday
Timely	• • • •	The same pupil's mark loaded twice

3 **A referential integrity test: find every mark whose pupil_id has no row in pupils. Show the pupil_id, subject and mark of each orphan.**

```
SELECT marks.pupil_id, subject, mark
FROM marks
JOIN pupils
  ON pupils.id = marks.pupil_id
WHERE pupils.id IS NULL;
```

Write your program:

4 **The contract says: subject is maths or science, and mark is a whole number from 0 to 20. problems(row) returns a list of reasons a row breaks it, but it misses one rule. Fix it, so the program prints every failure with its line number.**

```
import csv
SUBJECTS = ["maths", "science"]
def problems(row):
    out = []
    if row["subject"] not in SUBJECTS:
        out.append("unknown subject")
    if not row["mark"].isdigit():
        out.append("mark not a number")
    return out
with open("marks.csv") as f:
    rows = csv.DictReader(f)
    for n, row in enumerate(rows, 2):
        for p in problems(row):
            print("line", n, p)
```

Write your program:

5 **This range test flags 9 as out of range, which it isn't. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  marks = ["9", "25", "14"]
2  for m in marks:
3      if m > "20":
4          print(m, "out of range")
```

✎ **Which dataset do you depend on that someone else produces (a timetable, results, a club list)? What would you put in its contract?**

Check your answers

Data quality and contracts

Data engineering · Level 4 · Lesson 3

1 **Quarantine the 3 rows and alert the producer**

9,997 good rows flow on, and the 3 bad ones wait in quarantine, flagged, for the producer to fix at the source.

2 **Valid — A mark of 25 out of 20**

Unique — The same pupil's mark loaded twice

Consistent — A mark for a pupil who doesn't exist

Timely — Monday's file still missing on Wednesday

Each side needs its own test: range for validity, a duplicate-key query for uniqueness, an orphan check for consistency, and a freshness check on arrival time.

3

```
SELECT marks.pupil_id, subject, mark
FROM marks
LEFT JOIN pupils
  ON pupils.id = marks.pupil_id
WHERE pupils.id IS NULL;
```

LEFT JOIN keeps every mark; where no pupil matches, pupils.id is NULL.

4

```
import csv
SUBJECTS = ["maths", "science"]
def problems(row):
    out = []
    if row["subject"] not in SUBJECTS:
        out.append("unknown subject")
    if not row["mark"].isdigit():
        out.append("mark not a number")
    elif int(row["mark"]) > 20:
        out.append("mark out of range")
    return out
with open("marks.csv") as f:
    rows = csv.DictReader(f)
    for n, row in enumerate(rows, 2):
        for p in problems(row):
            print("line", n, p)
```

Each rule in the contract becomes one check with a reason.

5 **Line 3: `if int(m) > 20:`**

Comparing text compares letter by letter: "9" comes after "2", so "9" > "20" is true.